

SEP 09 – 11, 2025

CONTAINER
days
CONFERENCE

Getting Started with eBPF Made Easy



CONTAINERDAYS CONFERENCE 2025



Speakers

SEP 09 – 11, 2025

CONTAINER
days
CONFERENCE



Michael Friese

Senior Software Engineer, Microsoft

Maintainer, Inspektor Gadget ([@flyth](#))



Qasim Sarfraz

Software Engineer, Microsoft

Maintainer, Inspektor Gadget ([@mqasimsarfraz](#))

Agenda

- What is eBPF?
- Challenges
- Inspektor Gadget
 - Demo - Packet Counter Gadget
 - Demo - Metric Collection
 - Post processing / Other frontends
 - Roadmap
- Conclusion

What is eBPF?

- eBPF: extended Berkeley Packet Filter
- Runs sandboxed (eBPF) programs in the kernel.
- eBPF programs are event-driven and run when the kernel or an application reaches a hook point:
 - System calls
 - Function entry/exit
 - Kernel tracepoints
 - Network events



Example: Packet Counter

```
counter = 0
```

```
OnPacketSent():
```

```
    counter++
```

```
# Process in User Space (e.g CLI, Metrics Backend)
```

```
SendToUserSpace(counter)
```

Why should you care about eBPF?

Debugging

Application issues:

- Service Discovery (DNS)
- Connectivity issues (TCP)
- File Permissions (System call)

Monitoring

Identifying source of high latency:

- Slow DNS requests
- CPU profiling
- Excessive file read/write
- Network traffic

Security

Identifying potential malicious activity:

- Shell opening
- Binary changes in container image
- Filesystem access
- Filter network traffic

Challenges

Packaging & Orchestration

Distribute and run eBPF
program seamlessly
inside Kubernetes

Infrastructure & Workloads

Map low-level events to
constructs like
Kubernetes, containers,
and nodes

Observability Pipeline

Export enriched data
to Prometheus,
OpenTelemetry, or
other backends

SEP 09 – 11, 2025

CONTAINER days CONFERENCE



Kernel



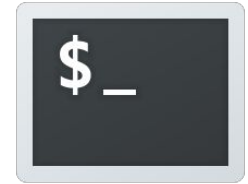
eBPF



WASM
User-Space
Programs



Kubernetes
Enrichment



OpenTelemetry



CLOUD NATIVE
COMPUTING FOUNDATION

Demo - Packet Counter Gadget

Goal: Write a Gadget to count packets sent per container

<https://github.com/mqasimsarfraz/gadget-packetcounter>

Packet Counter Gadget - Recording

SEP 09 – 11, 2025
CONTAINER
days
CONFERENCE

SEP 09 – 11, 2025
CONTAINER
days
CONFERENCE

Demo - Packet Counter Gadget

Goal: Write a Gadget to count packets sent per container

<https://github.com/mqasimsarfraz/gadget-packetcounter>

#CDS25

www.containerdays.io

Packet Counter Gadget - Summary

Why is this powerful?

- Provides a **familiar container-like developer experience** for eBPF developer.
- Makes it easy to **package, share, and orchestrate** eBPF programs.
- Scales from **local debugging** to **cluster-wide observability**.

Demo - Metric Collection

Goal: Send our freshly created per-container metrics to Prometheus

<https://github.com/mqasimsarfraz/gadget-packetcounter>

Metric Collection - Recording

Demo - Metric Collection

Goal: Send our freshly created per-container metrics to Prometheus

<https://github.com/mqasimsarfraz/gadget-packetcounter>

#CDS25

www.containerdays.io

Metric Collection - Summary

Why is this powerful:

- Supports **per-field metric collection** to control cardinality.
- Easily **extensible** by adding new fields.
- Compatible with OpenTelemetry:
 - Metrics
 - Logs



Post Processing

- eBPF is powerful for in-kernel

observability, but it has **strict**

limitations:

- number of instructions
 - limited set of instructions
- Performance trade-offs

Hello world gadget with Wasm

This guide explores the wasm support to implement complex logic in our gadget. This is a continuation of [hello world gadget](#), be sure you are familiar with that guide before continuing with this.

Creating our first wasm program

Create a folder named `go` next to the `program.bpf.c` file. In there we create a new file named `program.go`. As a first step, let's define the `init`, `start` and `stop` functions and emit some log messages from them:

```
package main

import (
    api "github.com/inspektor-gadget/inspektor-gadget/wasmapi/go"
)

//go:wasmexport gadgetInit
func gadgetInit() int32 {
    api.Info("init: hello from wasm")
    return 0
}

//go:wasmexport gadgetStart
func gadgetStart() int32 {
    api.Info("start: hello from wasm")
    return 0
}

//go:wasmexport gadgetStop
func gadgetStop() int32 {
    api.Info("stop: hello from wasm")
    return 0
}

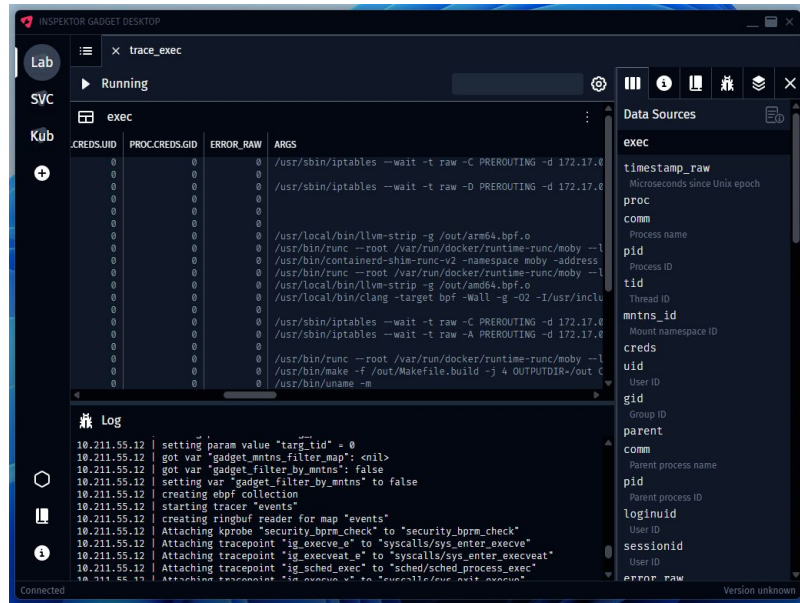
// The main function is not used, but it's still required by the compiler
func main() {}
```



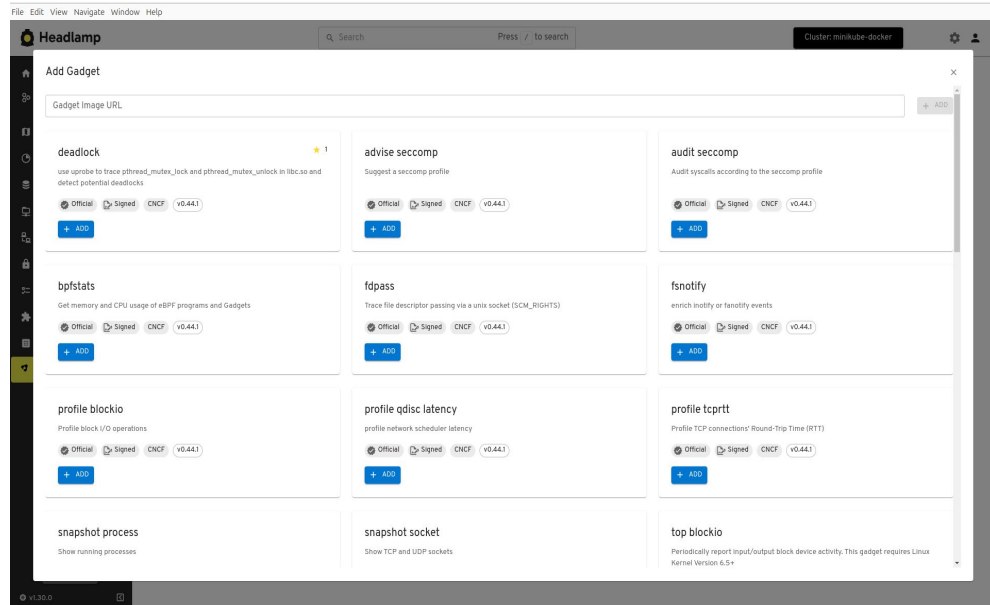
<https://inspektor-gadget.io/docs/latest/gadget-devel/hello-world-gadget-wasm/>

Other Frontends

IG Desktop



IG Headlamp Plugin



Roadmap

Framework

- Better user stack symbolization
- Generic packet filtering
- Enhanced OpenTelemetry support

Gadgets

- “tcpdump” (write to pcap files)
- Better metrics support for existing gadgets
- GPU metrics / debugging

Conclusion

Building a gadget ecosystem together

Official



trace dns
Inspektor Gadget Official gadgets

★ 0 Inspektor gadget
Updated 5 days ago
Version 0.44.1

trace dns requests and responses

Monitoring and logging



trace fsslower
Inspektor Gadget Official gadgets

★ 0 Inspektor gadget
Updated 5 days ago
Version 0.44.1

Community



syscall-count
dorser syscall count

★ 0 Inspektor gadget
Updated a year ago
Version 0.1.0

Collect count metrics for selected syscalls called within containers

Monitoring and logging



trace-udns
alban trace-udns

★ 0 Inspektor gadget
Updated a year ago
Version 0.1.0

Thank you!



Kubernetes-Slack

[#inspektor-gadget](#)

<https://inspektor-gadget.io>

<https://github.com/inspektor-gadget/inspektor-gadget>

